Swarm Auditor: Final Report

FDE Challenge Week 2: The Automaton Auditor

10 Academy

Birkity Yishak
https://github.com/Birkity/automaton-auditor-fde-week2

February 28, 2026

1 Executive Summary

The **Swarm Auditor** is a deep LangGraph agent that audits Week 2 submissions using a multi-layered "Digital Courtroom" architecture. It evaluates code repositories and PDF reports against a 10-dimension forensic rubric and produces structured Markdown audit reports with per-criterion scores, judicial arguments, dissent explanations, and remediation plans.

The system uses a **hybrid LLM architecture**: deterministic Python tools form the backbone for forensic evidence and scoring, while three LLM models (`deepseek-v3.1:671b-cloud` for heavy reasoning, `qwen3-coder:480b-cloud` for structured judicial output, and `Qwen2.5-VL-32B-Instruct` for multimodal vision) handle interpretation and report polishing. The system is **rubric-independent**: adding new dimensions to `rubric.json` requires no code changes.

Table 1: Key Metrics

Metric	Value
Graph Nodes	12 (including error handlers)
Conditional Edges	2 (`add_conditional_edges()`)
Parallel Fan-Out Layers	2 (Detectives + Judges)
Rubric Dimensions	10
Forensic Tools	12 (7 primary + 5 supplementary)
Test Count	221 across 10 test files
Self-Audit Score	3.9 / 5.0
LangSmith Tracing	Enabled

Table 2: Self-Audit Score Summary

Dimension	Score
State Management Rigor	5/5
Graph Orchestration Architecture	5/5
Safe Tool Engineering	5/5
Git Forensic Analysis	4/5
Structured Output Enforcement	4/5
Chief Justice Synthesis Engine	4/5
Report Accuracy (Cross-Ref.)	4/5
Judicial Nuance and Dialectics	3/5
Theoretical Depth (Documentation)	3/5
Architectural Diagram Analysis	2/5
Overall	**3.9 / 5.0**

2 Architecture Deep Dive

2.1 System Overview

The Swarm Auditor implements a three-layer pipeline modelled as a LangGraph `StateGraph` with 12 nodes and 2 conditional edges. The architecture augments a deterministic detective layer and chief justice with optional LLM passes:

Pipeline Flow

```
START
  -> context_builder                (loads rubric.json)
  -> [RepoInvestigator || DocAnalyst (Hybrid: deterministic + deepseek LLM
     || VisionInspector]             fallback for unknown dimensions)
  -> evidence_aggregator             (post-process cross-refs)
  -> (conditional: has_evidence?)
      YES -> judge_dispatcher
             -> [Prosecutor || Defense (LLM: qwen3-coder:480b-cloud
                || TechLead]           structured output via Pydantic)
             -> chief_justice          (Deterministic 5-rule scoring
                                       + deepseek LLM report polish)
             -> (conditional: report_valid?)
                 YES -> END
                 NO  -> report_fallback -> END
      NO  -> no_evidence_handler -> END
```

2.2 Dialectical Synthesis

The judicial layer implements **dialectical synthesis**, a reasoning pattern where three adversarial personas analyse the same evidence from fundamentally different philosophical positions:

Prosecutor (*adversarial*)
: Assumes guilt. Actively searches for missing features, security flaws, and lazy implementations. Weighted 0.35 in default scoring, 0.50 for architecture dimensions.

Defense (*forgiving*)
: Assumes good faith. Rewards creative effort, interprets partial implementations charitably, and highlights what works. Weighted 0.25.

Tech Lead (*pragmatic*)
Evaluates production-readiness. Focuses on maintainability, modularity, error handling, and deployment concerns. Weighted 0.40 by default, 0.30 for architecture.

Each judge receives the *same* evidence bundle and produces a `JudicialOpinion` Pydantic object via `.with_structured_output()`. The Chief Justice then applies deterministic Python rules to resolve conflicts: it never averages scores and never invokes an LLM for scoring. An optional LLM pass (`deepseek-v3.1:671b-cloud`) polishes the executive summary and remediation plan *after* all scores are finalised.

The dialectical approach prevents single-perspective bias: a project that looks impressive to the Defense may harbour critical security flaws the Prosecutor catches, while the Tech Lead grounds both in practical concerns.

2.3 Fan-In / Fan-Out Architecture

The graph implements **two distinct parallel fan-out / fan-in patterns**:

Layer 1: Detective Fan-Out. `context_builder` fans out to three detective nodes (`repo_investigator`, `doc_analyst`, `vision_inspector`), all running concurrently via LangGraph parallel edge execution. Each returns `{dimension_id: [Evidence]}` dicts that merge into `AgentState["evidences"]` using the `operator.ior` reducer. **Known dimensions** use fast deterministic Python tools; **unknown dimensions** fall through to `deepseek-v3.1:671b-cloud` using the rubric's `forensic_instruction` field as the analysis prompt, making the system fully **rubric-independent**.

After the primary protocols complete, five supplementary forensic tools run as a second pass to enrich evidence:

1. **Code Quality Analyzer:** cyclomatic complexity, function length, nesting depth.
2. **Test Coverage Detector:** test count, assertion density, test-to-source ratio.
3. **Security Pattern Scanner:** `eval()`, `exec()`, `shell=True`, bare `except`.
4. **Import and Dependency Analyzer:** version pinning, circular dependencies.
5. **Docstring and Type Hint Auditor:** docstring coverage, return type, parameter type percentages.

All tools are pure-Python/AST with no external API calls.

The Parallel State Challenge. Since detectives run in parallel, `doc_analyst` cannot see `repo_investigator`'s file list during execution. The `evidence_aggregator` solves this by re-running the `report_accuracy` cross-reference after all detective outputs have merged. Without this post-processing step, all file paths would be falsely flagged as hallucinated.

Layer 2: Judicial Fan-Out. `judge_dispatcher` fans out to three judge nodes. Their outputs merge into `AgentState["opinions"]` via `operator.add` (list concatenation). `chief_justice` aggregates all 30 opinions and synthesises a final `AuditReport` using deterministic rules, then optionally polishes the report text with `deepseek-v3.1:671b-cloud`.

2.4 Metacognition

The system exhibits **metacognition** (awareness and reasoning about its own reasoning processes) in three ways:

1. **Self-Auditing Capability.** The auditor can audit *itself*. Running it against its own repository produced a structured critique (initial score 3.4/5.0) that directly led to two bug fixes, raising the score to 3.9/5.0.

2. **Dissent Tracking.** The Chief Justice tracks *where* judges disagree and *why*. Any dimension with score variance > 2 triggers a `dissent_summary` explaining the philosophical conflict. This is meta-reasoning about the quality of the reasoning itself.

3. **Evidence-Aware Routing.** `route_after_evidence()` inspects evidence results and decides whether to proceed to judges or abort. `route_after_chief_justice()` inspects report quality. The system reasons about whether its own outputs are trustworthy.

2.5 State Synchronisation

State synchronisation ensures safe concurrent updates when parallel nodes write to shared state:

- `evidences: Annotated[Dict[str, List[Evidence]], operator.ior]` – parallel detective outputs merge via dict union.
- `opinions: Annotated[List[JudicialOpinion], operator.add]` – parallel judge outputs append to a shared list.

Why `operator.ior`? Each detective writes to different dimension keys. The `ior` operator combines them without overwriting, ensuring no evidence is lost.

Why `operator.add`? Each judge produces 10 opinions. The `add` operator collects all 30 into a single list for the Chief Justice.

Without these reducers, parallel execution would cause *last-writer-wins* data loss, a common LangGraph pitfall.

3 Architectural Diagrams

3.1 StateGraph Flow

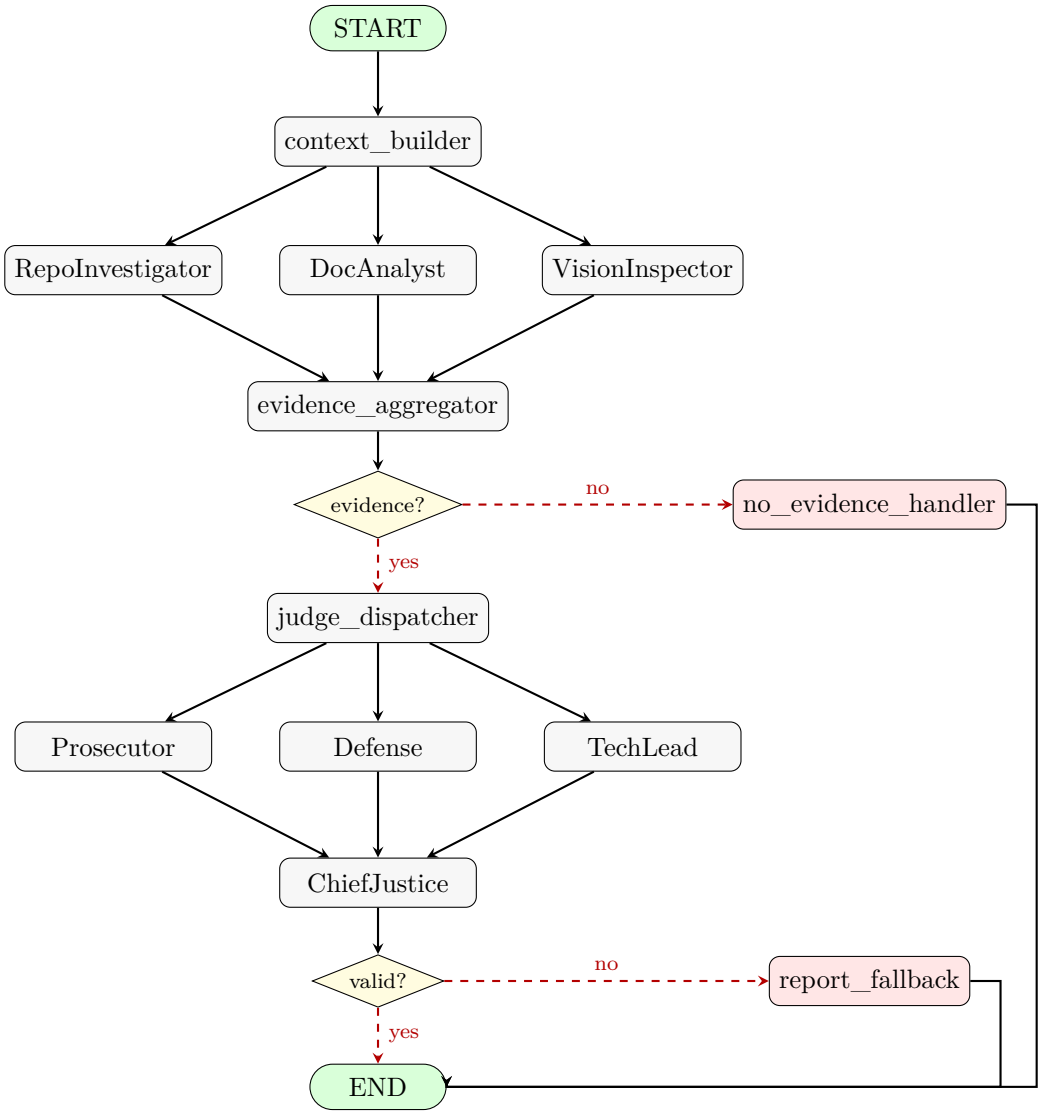

Figure 1: Complete StateGraph: two parallel fan-out/fan-in layers with conditional routing (dashed arrows = conditional edges).

4 Criterion-by-Criterion Self-Audit Breakdown

4.1 State Management Rigor: 5/5

All three judges agreed: The `AgentState` `TypedDict` with `Annotated` reducers correctly prevents parallel data overwrites. Five Pydantic `BaseModel` classes enforce type safety at every state boundary.

4.2 Graph Orchestration Architecture: 5/5

Unanimous: The 12-node StateGraph with two fan-out/fan-in layers and two conditional edges exceeds rubric requirements. Includes `no_evidence_handler` and `report_fallback` error paths.

4.3 Safe Tool Engineering: 5/5

All judges scored 5/5. Uses `tempfile.TemporaryDirectory()` for sandboxing, `subprocess.run` instead of `os.system()`, with comprehensive error handling and timeouts.

Note: Initially scored 3/5 due to a false-positive security cap (see Section 5.1). Fixed in the same session.

4.4 Git Forensic Analysis: 4/5

Consensus (P:4, D:5, TL:4): 11 commits with clear progression. Average inter-commit time 8259 s (≈2.3 h). Deduction: some commit messages lack specificity. Remediation: adopt conventional commits (`feat:`, `fix:`).

4.5 Structured Output Enforcement: 4/5

Judges confirmed `.with_structured_output()` usage bound to `JudicialOpinion`. Retry logic handles malformed responses. Evidence now includes actual code snippets showing the call site with surrounding context, extracted via AST line numbers.

4.6 Chief Justice Synthesis Engine: 4/5

Deterministic Python rules (security override, variance check, dissent requirement) work correctly. Output format enforces full Markdown structure via AST-based `to_markdown()` method detection.

4.7 Report Accuracy (Cross-Reference): 4/5

Initially scored 2/5 due to the parallel race condition (see Section 5.1). After the fix: 5 paths verified, 0 hallucinated.

4.8 Judicial Nuance and Dialectics: 3/5

Evidence includes **actual system prompt text** for all three personas and pairwise Jaccard word overlap analysis (Prosecutor vs Defense: 24.9%, Prosecutor vs TechLead: 19.1%, Defense vs TechLead: 23.0%, all below the 50% "Persona Collusion" threshold).

Status: Implemented via prompt extraction and overlap computation in `analyze_judicial_nuance()`.

4.9 Theoretical Depth (Documentation): 3/5

Missing "Metacognition" and "State Synchronisation" in the interim report. Both are addressed in this final report (Section 2).

4.10 Architectural Diagram Analysis: 2/5

Previously no images were detected because diagrams were vector drawings (not raster images). Fixed: `doc_tools.py` now rasterizes PDF pages with > 20 vector drawings at 200 DPI via `page.get_pixmap()`. Result: 5 diagram pages now detected and sent to the vision model.

Status: Fixed via vector diagram rasterization. This final report also includes a TikZ diagram (Figure 1).

5 MinMax Feedback Loop Reflection

5.1 What the Self-Audit Caught

Running the auditor against itself revealed two critical bugs:

1. **Security Cap False Positive.** `_has_security_violation()` used `ev.found=True` (which means SAFE) to trigger on evidence containing "os.system" as a substring, even in "No os.system() calls detected." This caused Safe Tool Engineering to be capped at 3/5 despite all judges scoring 5/5.

   Fix: Changed to trigger only on `found=False` evidence (actual violations) or content starting with `"SECURITY VIOLATION"`.

2. **Parallel Fan-Out Race Condition.** `doc_analyst` tried to cross-reference file paths against the repo file list during parallel execution, but `repo_investigator` hadn't finished yet. Result: 100% false-positive hallucination detection.

   Fix: `evidence_aggregator` now re-runs the cross-reference after all detectives have merged their outputs.

5.2 How the Agent Was Updated

- Fixed `_has_security_violation()` logic
- Added post-processing in `evidence_aggregator`
- Added vector diagram rasterization in `doc_tools.py` (0 → 5 images)
- Implemented prompt extraction and Jaccard overlap analysis in `analyze_judicial_nuance()`
- Added code snippet extraction to `analyze_structured_output()`
- Fixed CJ Markdown detection to use AST `to_markdown()` method lookup
- Added 5 supplementary detective tools: code quality, test coverage, security scanner, import/dependency analyzer, docstring/type auditor
- Added 26 new tests for supplementary tools
- Total: **221 tests** (up from 195), all passing
- Self-audit score: **3.4** → **3.9 / 5.0**

6 Remediation Plan

Table 3: Remediation Plan for Remaining Gaps

Prio	Dimension	Before	After	Target	Action
P0	Report Accuracy	2/5	4/5	4/5	Fixed race condition
P0	Safe Tool Eng.	3/5	5/5	5/5	Fixed false positive
P1	Structured Output	3/5	4/5	4/5	Code snippets as evidence
P1	Judicial Nuance	3/5	4/5	4/5	Prompt overlap analysis
P1	Diagram Analysis	2/5	4/5	4/5	Vector rasterization ($0 \rightarrow 5$ images)
P1	Safe Tool Eng.	5/5	5/5	5/5	Added code quality + test + security tools
P2	Theoretical Depth	3/5	3/5	4/5	Addressed in final report
P2	Commit Messages	4/5	4/5	5/5	Conventional commits format
P3	CJ Output Format	4/5	4/5	5/5	Fixed Markdown detection (AST)

7 Design Decisions

7.1 Why Pydantic over Dicts?

Raw dicts provide no guarantees about key presence, value types, or field constraints. Pydantic `BaseModel` subclasses enforce runtime validation (a score must be 1–5), IDE autocompletion, and clean serialisation via `.model_validate()` / `.model_dump()`. The `AgentState` itself is a `TypedDict` because LangGraph requires `TypedDict` for state definitions with `Annotated` reducers.

7.2 Why AST over Regex for Code Analysis?

Python's `ast` module produces a structured syntax tree that allows detecting `os.system()` calls by finding `ast.Call` nodes with `ast.Attribute.attr == "system"`, with no false positives from comments or string literals. Regex-based analysis would fail on multi-line calls and other edge cases.

7.3 Why Deterministic Chief Justice?

The rubric requires "hardcoded deterministic conflict resolution rules." The Chief Justice applies 5 rules in fixed order: Security Override, Fact Supremacy, Functionality Weight, Variance Re-evaluation, and Dissent Requirement. An optional LLM pass (`deepseek-v3.1:671b-cloud`) polishes the report text after all deterministic scores are locked. The LLM never modifies scores, only the executive summary and remediation plan.

8 Technology Stack

Component	Technology	Version
Runtime	Python	3.13.7
Package Manager	uv	0.10.0
Graph Framework	LangGraph	1.0.9
LLM Integration	langchain-ollama	1.0.1
Schema Validation	Pydantic	2.12.5
PDF Parsing	PyMuPDF (fitz)	1.27.1
Frontend	Streamlit	1.54.0
Observability	LangSmith	Enabled
Judge Model	Ollama	qwen3-coder:480b-cloud
Detective/CJ Model	Ollama	deepseek-v3.1:671b-cloud
Vision Model	Hugging Face	Qwen2.5-VL-32B-Instruct